

Activity 4: SQL Query Generation and Optimization using AI

Purpose

Learn how to use AI to understand database requirements, generate SQL queries, optimize query performance, and validate AI-generated SQL before using it in an application.

Pre-requisites

- Completion of Activities 1, 2 and 3
- VS Code with Cline configured
- Basic understanding of SQL statements

Scenario

The SmartBank backend team has designed a database to store loan details submitted through the EMI Calculator. As a developer, you need to use AI to understand the database structure, generate SQL queries, optimize them, and validate the generated SQL before sharing it with the database team.

Note: The objective of this activity is to use AI for SQL generation and optimization. Connecting to an actual database or executing SQL statements is outside the scope of this activity.

Tasks

Task 1: Understand the Database Schema

Assume the following table has been created.

Loan Table

Column	Data Type
Loan_ID	NUMBER
Customer_Name	VARCHAR2(100)
Loan_Type	VARCHAR2(30)
Loan_Amount	NUMBER
Interest_Rate	NUMBER
Loan_Tenure	NUMBER
Monthly_EMI	NUMBER
Created_Date	DATE

Analyze the schema before generating SQL.

Suggested Prompt

Act as a Database Developer.

Analyze the following Loan table schema.

Explain:

- Purpose of the table
- Purpose of each column
- Primary key recommendation
- Possible constraints
- Any improvements to the schema
- Best practices for storing loan information

Explain your recommendations before generating any SQL.

Task 2: Generate SQL Queries

Generate commonly used queries for the Loan table.

Suggested Prompt

Using the Loan table schema, generate SQL queries for the following business requirements:

- Retrieve all loan records.
- Retrieve a loan using Loan_ID.
- Retrieve loans by Loan_Type.
- Retrieve loans where Loan_Amount is greater than a specified value.
- Retrieve loans created today.
- Sort loan records by Loan_Amount in descending order.

Use clear formatting and explain the purpose of each query.

Task 3: Generate Advanced SQL Queries

Generate analytical queries that may be useful for reporting.

Suggested Prompt

Generate SQL queries for the following reporting requirements:

- Total number of loans.
- Average loan amount.
- Highest monthly EMI.
- Total loan amount grouped by Loan_Type.
- Number of loans created each day.

Explain how each query works.

Task 4: Optimize the SQL Queries

Review the generated SQL and suggest improvements.

Suggested Prompt

Review the generated SQL queries as a Database Performance Engineer.

Evaluate:

- Query readability
- Performance
- Opportunities to simplify the queries
- Index recommendations
- SQL best practices

Explain each recommendation.

Do not modify the table structure.

Task 5: Validate AI-Generated SQL

Review the generated SQL before using it.

Suggested Prompt

Review the generated SQL queries.

Identify:

- Logical errors
- Missing conditions
- Possible performance issues
- Incorrect assumptions
- Opportunities to improve maintainability

Provide a validation report without rewriting the queries.

Activity 5: Validate and Review AI-Generated Code

Purpose

Learn how to critically evaluate AI-generated code for correctness, readability, maintainability, performance, and alignment with enterprise development standards before using it in a software project.

Pre-requisites

- Completion of Activities 1–4
- Working EMI Calculator application
- VS Code with Cline configured

Scenario

During the previous sprint, your team used AI to generate application code, refactor existing logic, prepare unit tests, generate SQL queries, and draft API integration code.

Before the application can move to the next phase, the Solution Architect has requested a comprehensive review of all AI-generated artifacts to ensure they meet enterprise quality standards.

As a developer, your task is to use AI to review, validate, and improve the generated outputs before they are accepted into the project.

Note: AI-generated code should always be reviewed and validated. Developers remain responsible for ensuring the correctness, security, maintainability, and overall quality of the final implementation.

Tasks

Task 1: Review the Application Code

Analyze the Oracle JET application for overall code quality.

Suggested Prompt

Act as a Senior Software Architect.

Review the current Oracle JET EMI Calculator application.

Evaluate:

- Overall code quality
- Readability
- Maintainability
- Naming conventions
- Modularity
- Reusability
- Oracle JET and KnockoutJS best practices

Identify strengths, weaknesses, and improvement opportunities.

Do not modify the code.

Task 2: Validate Business Logic

Ensure the application implements the required business rules correctly.

Suggested Prompt

Review the EMI calculation logic implemented in this application.

Validate:

- Mathematical correctness
- Input validation
- Boundary conditions
- Error handling
- Business assumptions

Identify any logical issues or missing scenarios.

Explain your findings before suggesting improvements.

Task 3: Review Previously Generated Artifacts

Review all AI-generated outputs created during the previous activities.

Suggested Prompt

Review the AI-generated artifacts created for this project, including:

- Refactored application code
- Unit test suggestions
- Mock test data
- API integration code
- SQL queries

For each artifact evaluate:

- Correctness
- Completeness
- Readability
- Maintainability
- Enterprise best practices

Summarize your findings in a review report.

Task 4: Generate a Quality Assessment Report

Prepare a report summarizing the readiness of the application.

Suggested Prompt

Prepare a quality assessment report for the current EMI Calculator project.

Include:

- Code quality assessment
- Testing readiness
- API integration readiness
- SQL quality review
- Maintainability assessment
- Risks identified
- Recommendations before release

Present the report in a professional format suitable for a project review meeting.